CLASS XII

INFORMATICS PRACTICES

PROJECT REPORT

ON

E-COMMERCE MANAGEMENT SYSTEM

(Session 2026 - 2027)

Submitted by:	**Submitted to:**
NAME: B.A.Kanishar Raj Class: XII C Roll No:	Name: K.Vimala Thomas PGT (Informatics Practices) School Name: PM SHRI KENDRIYA VIDYALAYA NAGERCOIL

CERTIFICATE

This is to certify that **B.A.KANISHAR RAJ** , a student of Class XII, has successfully completed the project entitled "***E-Commerce Management System***" in Informatics Practices during the academic year 2026-2027, under my guidance and supervision.

The project has been carried out as part of the CBSE Class XII Informatics Practices curriculum and is found to be satisfactory.

This project is genuine and has been completed as per the guidelines issued by the Central Board of Secondary Education (CBSE).

Internal Examiner Date: _____	External Examiner Date: _____

Principal's Signature and School Seal: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my Informatics Practices teacher **K.VIMALA THOMAS** for her valuable guidance, encouragement and support throughout the completion of this project, "***E-Commerce Management System***."

I am also thankful to our Principal for providing the necessary facilities and a conducive environment to carry out this project work.

I extend my heartfelt thanks to my parents and friends for their constant motivation and support during the course of this project.

Finally, I would like to thank the Central Board of Secondary Education (CBSE) for including such a practical and skill-based project in the Class XII Informatics Practices curriculum, which helped me understand real-life applications of Python programming.

DECLARATION

I hereby declare that the project entitled "***E-Commerce Management System***" is an original piece of work done by me under the guidance of my Informatics Practices teacher.

This project has been developed using the concepts of Python programming taught as part of the Class XII Informatics Practices syllabus and has not been copied from any other source.

The information and code presented in this report are true and correct to the best of my knowledge.

Name: B.A.KANISHAR RAJ

Class: XII C

Roll No:

INDEX

S.No.	Topic	Page No.
1	Certificate	2
2	Acknowledgement	3
3	Declaration	4
4	Introduction to E-Commerce	6
5	Need for E-Commerce Management System	7
6	Objectives of the Project	8
7	Scope of the Project	9
8	Hardware and Software Requirements	10
9	Python Concepts Used	11
10	Data Structures Used	13

11	System Design / Working of the System	14
12	Product Data	15
13	Customer and Cart Management	16
14	Menu and Program Flow	17
15	Complete Python Source Code	18
16	Explanation of Important Code Sections	25
17	Sample Input / Output	27
18	Data Visualization - Bar Chart	28
19	Data Visualization - Line Graph	29
20	Data Visualization - Histogram	30
21	Testing, Advantages, Limitations & Future Scope	31

22	Conclusion and Bibliography	33

How This Project Uses Class XII IP Concepts

This project has been created using only the concepts that are taught in the Class XII Informatics Practices syllabus. It uses Python programming basics, Lists, Dictionaries, Functions, conditional statements, loops, simple exception handling and the Matplotlib library for data visualization (bar chart, line graph and histogram). No advanced topics such as Object Oriented Programming, databases, pandas, numpy, web development or APIs have been used, since these are outside the scope of the current syllabus.

Introduction to E-Commerce

E-Commerce, short for Electronic Commerce, refers to the buying and selling of goods and services over the internet. In today's digital world, e-commerce has become an important part of everyday life. People can order groceries, clothes, electronics, books and many other products online without having to visit a physical shop.

An E-Commerce Management System is a software application that helps a business manage its products, customers, shopping cart, orders and sales in an organized way. Such a system makes it easy to keep track of what is being sold, to whom, and for how much.

This project is a simplified version of such a system, created using basic Python programming concepts that are taught in Class XII. It is menu driven, which means the user is shown a list of options and the program performs the task selected by the user.

Need for E-Commerce Management

As more businesses move online, there is a growing need for simple systems that can manage day-to-day store operations. An E-Commerce Management System is needed because:

- It keeps a proper record of all available products along with their price and stock.
- It allows customers to search for products quickly instead of going through the entire list.
- It maintains a shopping cart so that customers can collect items before making a final purchase.
- It automatically calculates the total bill and applies discounts wherever applicable.
- It stores customer details and order history for future reference.
- It helps in analysing sales data with the help of graphs, so that better business decisions can be made.

Without such a system, all these tasks would have to be done manually, which is time consuming and prone to errors.

Objectives of the Project

The main objectives of this project are:

- To design a simple, menu-driven Python program for managing an online store.
- To apply the concepts of Lists, Dictionaries and Functions learnt in the IP syllabus.
- To allow users to view, search and purchase products through a cart system.
- To calculate bills automatically, including discount rules.
- To store and display customer and order information.
- To visualize sales data using Matplotlib graphs (bar chart, line graph and histogram).
- To strengthen programming and logical thinking skills through a real-life application.

Scope of the Project

This project is designed as a learning tool to demonstrate how basic Python programming concepts can be applied to a real-world scenario like online shopping. The scope of the project includes:

- Displaying and searching products stored in the system.
- Adding products to a cart and viewing the cart contents.
- Calculating the total bill with a simple discount policy.
- Recording customer details and placing orders.
- Displaying past order details during the same program run.
- Showing basic sales trends using graphs.

This project does not include real payment gateways, login/signup systems, permanent database storage or internet connectivity, as these concepts are beyond the scope of the Class XII IP syllabus. All the data used is sample data stored using Lists and Dictionaries which resets every time the program is run.

Hardware and Software Requirements

Hardware Requirements

Component	Minimum Requirement
Processor	Dual Core Processor or above
RAM	2 GB or above
Hard Disk	20 GB free space
Monitor	Standard Monitor
Keyboard/Mouse	Standard Keyboard and Mouse

Software Requirements

Component	Requirement
Operating System	Windows 10 / Windows 11 / Linux / macOS
Programming Language	Python 3.x
IDE	Python IDLE / any Python editor
Library Used	matplotlib

Python Concepts Used

This project uses only those Python concepts that are a part of the Class XII Informatics Practices syllabus. For every concept, its meaning, purpose in this project, and an example from the code are given below.

Variables

Use in project: Used to store data values such as product price, quantity and total amount.

```
Example: total = total + (item["price"] * item["qty"])
```

Input/Output

Use in project: Used to take input from the user (like product ID) and display results.

```
Example: pid = int(input("Enter Product ID to add to cart: "))
```

if / elif / else

Use in project: Used to make decisions, such as applying discount based on bill amount.

```
Example: if total >= 20000: discount = total * 0.10
```

for loop

Use in project: Used to go through each product/item in a list.

```
Example: for p in products: print(p["name"])
```

while loop

Use in project: Used to keep showing the menu until the user chooses to exit.

```
Example: while True: ... (menu code)
```

Functions

Use in project: Used to break the program into smaller, reusable blocks like display_products(), add_to_cart().

```
Example: def add_to_cart(): ...
```

Lists

Use in project: Used to store multiple products, cart items and orders together.

```
Example: products = [ {...}, {...}, {...} ]
```

Dictionaries

Use in project: Used to store related information together, like one product's id, name, price and stock.

```
Example: {"id":101, "name":"Laptop", "price":45000}
```

Exception Handling

Use in project: Used to handle invalid input, such as entering text instead of a number.

```
Example: try: pid = int(input(...)) except ValueError: print("Invalid input")
```

Matplotlib

Use in project: Used to create bar chart, line graph and histogram for sales data.

```
Example: plt.bar(categories, sales)
```

Data Structures Used

The following simple data structures have been used to organize the information handled by this project:

1. List of Dictionaries (Products)

All products are stored in a list named 'products'. Each product itself is a dictionary containing keys like id, name, category, price and stock. This makes it easy to store many products together while keeping their individual details organized.

2. List (Cart and Orders)

The 'cart' and 'orders' are also lists. Every time a product is added to the cart, a new dictionary with its details is appended to the cart list. Similarly, every order placed is appended to the orders list.

3. Dictionary (Customer and Sales Data)

The 'customer' dictionary stores the name, phone number and address of the customer. The 'category_sales' dictionary stores category names as keys and total units sold as values, which is later used to plot the bar chart.

System Design / Working of the System

The E-Commerce Management System works in a simple step-by-step manner, as described below:

- The program starts and displays a menu with 12 options.
- The user enters a number to choose an option.

- Based on the choice, the corresponding function is called (for example, choice 1 calls display_products()).
- The user can browse products, search for a product, and add products to the cart.
- The cart keeps track of items until the user calculates the bill or places the order.
- Once the customer details are entered and the order is placed, the order is saved and the cart is emptied.
- The user can view sales information and graphs at any time.
- The program keeps repeating the menu until the user selects the Exit option.

This flow ensures that the program behaves like a simplified real-life online shopping system, while still remaining easy to understand for a Class XII student.

Product Data

The system contains 8 sample products divided into four categories: Electronics, Furniture, Books and Fashion. Each product is stored as a dictionary with the following keys: id, name, category, price and stock. The table below shows the initial product data used in the program:

ID	Name	Category	Price (Rs.)	Stock
101	Laptop	Electronics	45000	10
102	Mobile Phone	Electronics	15000	20
103	Study Table	Furniture	3500	15
104	Office Chair	Furniture	2500	12
105	Story Book	Books	250	50
106	Notebook Set	Books	150	60
107	Sports Shoes	Fashion	2200	25
108	T-Shirt	Fashion	600	40

Customer and Cart Management

Customer Management: The program stores customer information (name, phone number and address) using a dictionary named 'customer'. This information is collected using the enter_customer_details() function and displayed using display_customer_details().

Cart Management: The 'cart' is a list that stores every product a customer wishes to purchase. When a product is added, a dictionary containing its id, name, price and quantity is appended to the cart list, and the stock of that product is reduced accordingly. The user can view the cart at any time using the view_cart() function. Once an order is placed, the cart is emptied using the clear() method so that a new set of items can be added for the next order.

This design keeps the customer and cart handling logically separate, while still working together smoothly when an order is placed.

Menu and Program Flow

The program uses a while loop to continuously display the main menu until the user chooses to exit. The menu options and the function called for each option are listed below:

Choice	Menu Option	Function Called
1	Display Products	display_products()
2	Search Product	search_product()
3	Add Product to Cart	add_to_cart()
4	View Cart	view_cart()
5	Calculate Total Bill	calculate_bill()
6	Enter Customer Details	enter_customer_details()
7	Display Customer Details	display_customer_details()
8	Place Order	place_order()
9	Display Order Details	display_orders()
10	Display Sales Information	display_sales_info()

11	Show Graphs	graph_menu()
12	Exit	break (ends the while loop)

Complete Python Source Code

The complete, working Python program for the E-Commerce Management System is given below. The code is divided into clearly commented sections: data storage, function definitions and the main menu-driven program.

```
"""
PROJECT     : E-COMMERCE MANAGEMENT SYSTEM
CLASS       : XII - INFORMATICS PRACTICES
DESCRIPTION : A menu driven Python program to manage products, cart,
              customers, orders and sales of a simple online store.
              This program uses Lists, Dictionaries, Functions and
              Matplotlib for data visualization.
"""

import matplotlib.pyplot as plt

# ------------------------------------------------------------
# GLOBAL DATA (Lists and Dictionaries used to store information)
# ------------------------------------------------------------

# Each product is stored as a dictionary. All products are kept in a list.
products = [
    {"id": 101, "name": "Laptop", "category": "Electronics", "price": 45000, "stock": 10},
    {"id": 102, "name": "Mobile Phone", "category": "Electronics", "price": 15000, "stock": 20},
    {"id": 103, "name": "Study Table", "category": "Furniture", "price": 3500, "stock": 15},
    {"id": 104, "name": "Office Chair", "category": "Furniture", "price": 2500, "stock": 12},
    {"id": 105, "name": "Story Book", "category": "Books", "price": 250, "stock": 50},
    {"id": 106, "name": "Notebook Set", "category": "Books", "price": 150, "stock": 60},
    {"id": 107, "name": "Sports Shoes", "category": "Fashion", "price": 2200, "stock": 25},
    {"id": 108, "name": "T-Shirt", "category": "Fashion", "price": 600, "stock": 40},
]

# Cart is a list of dictionaries. Every item added to the cart is stored here.
cart = []

# Customer information is stored using a dictionary.
customer = {"name": "", "phone": "", "address": ""}

# Sample sales data used only for graphs (category-wise sales, in units).
category_sales = {
    "Electronics": 320,
    "Furniture": 150,
    "Books": 210,
    "Fashion": 180,
}

# Sample month-wise sales data (in thousand rupees) used for the line graph.
month_names = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
month_sales = [120, 135, 150, 130, 160, 175]

# Sample past order values used for the histogram.
past_order_values = [500, 1200, 3000, 4500, 2200, 800, 6000, 15000,
                     2500, 3200, 1800, 9000, 700, 5200, 4100, 2600]
```

```
# This list stores all the orders placed during the current run of program.
orders = []


# ----------------------------------------------------------------
# FUNCTION DEFINITIONS
# ----------------------------------------------------------------

def display_products():
    """Displays all the products available in the store."""
    print("\n------ PRODUCT LIST ------")
    print("ID\tName\t\tCategory\tPrice\tStock")
    for p in products:
        print(p["id"], "\t", p["name"], "\t\t", p["category"],
              "\t", p["price"], "\t", p["stock"])


def search_product():
    """Searches a product by name entered by the user."""
    key = input("Enter product name to search: ")
    found = False
    for p in products:
        if key.lower() in p["name"].lower():
            print("Found:", p)
            found = True
    if not found:
        print("No product found with this name.")


def find_product_by_id(pid):
    """Returns the product dictionary matching the given id, else None."""
    for p in products:
        if p["id"] == pid:
            return p
    return None


def add_to_cart():
    """Adds a product to the cart list after checking stock."""
    display_products()
    try:
        pid = int(input("Enter Product ID to add to cart: "))
    except ValueError:
        print("Invalid input. Please enter a numeric ID.")
        return

    product = find_product_by_id(pid)
    if product is None:
        print("Product not found.")
        return

    qty = int(input("Enter quantity: "))
    if qty <= 0:
        print("Quantity must be greater than 0.")
        return
    if qty > product["stock"]:
        print("Sorry, only", product["stock"], "units available in stock.")
        return

    # Add a copy of details into the cart as a dictionary
    cart_item = {"id": product["id"], "name": product["name"],
                 "price": product["price"], "qty": qty}
    cart.append(cart_item)
    product["stock"] = product["stock"] - qty
```

```
    print(product["name"], "added to cart successfully.")


def view_cart():
    """Displays all items currently present in the cart."""
    if len(cart) == 0:
        print("\nYour cart is empty.")
        return
    print("\n------ YOUR CART ------")
    print("Name\t\tPrice\tQty\tSubtotal")
    for item in cart:
        subtotal = item["price"] * item["qty"]
        print(item["name"], "\t", item["price"], "\t", item["qty"], "\t", subtotal)


def calculate_bill():
    """Calculates total bill of the cart and applies discount if applicable."""
    if len(cart) == 0:
        print("\nCart is empty. Add products first.")
        return 0

    total = 0
    for item in cart:
        total = total + (item["price"] * item["qty"])

    # Simple discount rule (basic if-elif-else, as used in Class 12 IP)
    discount = 0
    if total >= 20000:
        discount = total * 0.10        # 10 percent discount
    elif total >= 10000:
        discount = total * 0.05        # 5 percent discount
    else:
        discount = 0

    final_amount = total - discount

    print("\n------ BILL SUMMARY ------")
    print("Total Amount     : Rs.", total)
    print("Discount Applied : Rs.", round(discount, 2))
    print("Final Amount     : Rs.", round(final_amount, 2))
    return round(final_amount, 2)


def enter_customer_details():
    """Takes and stores customer information using a dictionary."""
    customer["name"] = input("Enter your name: ")
    customer["phone"] = input("Enter your phone number: ")
    customer["address"] = input("Enter your address: ")
    print("Customer details saved successfully.")


def display_customer_details():
    """Displays the stored customer information."""
    if customer["name"] == "":
        print("\nNo customer details found. Please enter details first.")
    else:
        print("\n------ CUSTOMER DETAILS ------")
        print("Name    :", customer["name"])
        print("Phone   :", customer["phone"])
        print("Address :", customer["address"])


def place_order():
    """Places an order using the current cart and customer details."""
```

```
    if customer["name"] == "":
        print("\nPlease enter customer details before placing an order.")
        return
    if len(cart) == 0:
        print("\nCart is empty. Add products before placing an order.")
        return

    final_amount = calculate_bill()
    order_no = len(orders) + 1

    # Each order is stored as a dictionary inside the orders list
    order = {
        "order_no": order_no,
        "customer": customer["name"],
        "items": list(cart),      # copy of the cart list
        "amount": final_amount,
    }
    orders.append(order)

    print("\nOrder placed successfully! Your Order No. is", order_no)

    # Empty the cart after placing the order
    cart.clear()


def display_orders():
    """Displays details of all orders placed so far."""
    if len(orders) == 0:
        print("\nNo orders have been placed yet.")
        return
    print("\n------ ORDER DETAILS ------")
    for order in orders:
        print("\nOrder No :", order["order_no"])
        print("Customer :", order["customer"])
        print("Amount   : Rs.", order["amount"])
        print("Items    :")
        for item in order["items"]:
            print("   -", item["name"], "x", item["qty"])


def display_sales_info():
    """Displays simple sales information (category wise units sold)."""
    print("\n------ CATEGORY WISE SALES (Sample Data) ------")
    for category in category_sales:
        print(category, ":", category_sales[category], "units")


def show_bar_chart():
    """Bar chart showing category wise sales."""
    categories = list(category_sales.keys())
    sales = list(category_sales.values())

    plt.bar(categories, sales, color="skyblue")
    plt.title("Category-wise Sales")
    plt.xlabel("Product Category")
    plt.ylabel("Units Sold")
    plt.savefig("bar_chart.png")
    plt.show()


def show_line_graph():
    """Line graph showing sales over different months."""
    plt.plot(month_names, month_sales, color="green", marker="o", label="Sales (in '000 Rs.)")
    plt.title("Month-wise Sales Trend")
```

```
    plt.xlabel("Month")
    plt.ylabel("Sales (in thousand Rs.)")
    plt.legend()
    plt.savefig("line_graph.png")
    plt.show()


def show_histogram():
    """Histogram showing distribution of order values."""
    plt.hist(past_order_values, bins=5, color="orange", edgecolor="black")
    plt.title("Distribution of Order Values")
    plt.xlabel("Order Value (Rs.)")
    plt.ylabel("Number of Orders")
    plt.savefig("histogram.png")
    plt.show()


def graph_menu():
    """Sub-menu for displaying different graphs."""
    print("\n---- GRAPH MENU ----")
    print("1. Bar Chart (Category wise Sales)")
    print("2. Line Graph (Month wise Sales)")
    print("3. Histogram (Order Value Distribution)")
    print("4. Back to Main Menu")
    ch = input("Enter your choice: ")
    if ch == "1":
        show_bar_chart()
    elif ch == "2":
        show_line_graph()
    elif ch == "3":
        show_histogram()
    elif ch == "4":
        return
    else:
        print("Invalid choice.")


# ------------------------------------------------------------
# MAIN PROGRAM (Menu Driven using while loop)
# ------------------------------------------------------------

def main():
    while True:
        print("\n====================================")
        print("     E-COMMERCE MANAGEMENT SYSTEM")
        print("====================================")
        print("1.  Display Products")
        print("2.  Search Product")
        print("3.  Add Product to Cart")
        print("4.  View Cart")
        print("5.  Calculate Total Bill")
        print("6.  Enter Customer Details")
        print("7.  Display Customer Details")
        print("8.  Place Order")
        print("9.  Display Order Details")
        print("10. Display Sales Information")
        print("11. Show Graphs")
        print("12. Exit")

        choice = input("Enter your choice (1-12): ")

        if choice == "1":
            display_products()
        elif choice == "2":
```

```
            search_product()
        elif choice == "3":
            add_to_cart()
        elif choice == "4":
            view_cart()
        elif choice == "5":
            calculate_bill()
        elif choice == "6":
            enter_customer_details()
        elif choice == "7":
            display_customer_details()
        elif choice == "8":
            place_order()
        elif choice == "9":
            display_orders()
        elif choice == "10":
            display_sales_info()
        elif choice == "11":
            graph_menu()
        elif choice == "12":
            print("Thank you for shopping with us. Goodbye!")
            break
        else:
            print("Invalid choice. Please enter a number between 1 and 12.")


# Program execution starts here
if __name__ == "__main__":
    main()
```

Explanation of Important Code Sections

1. Storing Products

```
products = [
    {"id": 101, "name": "Laptop", "category": "Electronics", "price": 45000, "stock": 10},
    ...
]
```

Each product is a dictionary with keys like id, name, price and stock. All such dictionaries are kept together in a list called 'products', so the whole catalogue can be accessed easily using a for loop.

2. Adding a Product to the Cart

```
cart_item = {"id": product["id"], "name": product["name"],
             "price": product["price"], "qty": qty}
cart.append(cart_item)
product["stock"] = product["stock"] - qty
```

A new dictionary is created for the selected product and quantity, and it is added to the cart list using append(). The stock of the original product is also reduced by the quantity purchased.

3. Calculating the Bill with Discount

```
if total >= 20000:
```

```
    discount = total * 0.10
elif total >= 10000:
    discount = total * 0.05
else:
    discount = 0
```

This if-elif-else structure checks the total bill amount and decides how much discount to apply. A higher bill amount gets a higher discount percentage.

4. Handling Invalid Input

```
try:
    pid = int(input("Enter Product ID to add to cart: "))
except ValueError:
    print("Invalid input. Please enter a numeric ID.")
    return
```

The try-except block is used so that if the user enters something that is not a number, the program shows a proper message instead of crashing.

5. Menu Driven Structure

```
while True:
    ...
    choice = input("Enter your choice (1-12): ")
    if choice == "1":
        display_products()
    ...
    elif choice == "12":
        break
```

The while True loop keeps the menu running repeatedly. Based on the user's choice, the matching function is called using if-elif statements. Choosing option 12 breaks out of the loop and ends the program.

Sample Input / Output

Below is a sample run of the program showing a few menu operations:

```
==================================
 E-COMMERCE MANAGEMENT SYSTEM
==================================
1. Display Products
2. Search Product
3. Add Product to Cart
4. View Cart
5. Calculate Total Bill
6. Enter Customer Details
7. Display Customer Details
8. Place Order
9. Display Order Details
10. Display Sales Information
11. Show Graphs
12. Exit

------ PRODUCT LIST ------
ID    Name            Category  Price       Stock
101   Laptop          Electronics     45000     10
102   Mobile Phone            Electronics     15000     20
103   Study Table         Furniture       3500      15
104   Office Chair        Furniture       2500      12
105   Story Book          Books     250       50
106   Notebook Set            Books     150       60
107   Sports Shoes            Fashion   2200      25
108   T-Shirt         Fashion   600       40
```

==========

E-COMMERCE MANAGEMENT SYSTEM

==========

1. Display Products
2. Search Product
3. Add Product to Cart
4. View Cart
5. Calculate Total Bill
6. Enter Customer Details
7. Display Customer Details
8. Place Order
9. Display Order Details
10. Display Sales Information
11. Show Graphs
12. Exit

Found: {'id': 101, 'name': 'Laptop', 'category': 'Electronics', 'price': 45000, 'stock': 10}

==========

E-COMMERCE MANAGEMENT SYSTEM

==========

1. Display Products
2. Search Product
3. Add Product to Cart
4. View Cart
5. Calculate Total Bill
6. Enter Customer Details
7. Display Customer Details
8. Place Order
9. Display Order Details
10. Display Sales Information
11. Show Graphs
12. Exit

------ PRODUCT LIST ------

ID	Name	Category	Price	Stock
101	Laptop	Electronics	45000	10
102	Mobile Phone	Electronics	15000	20
103	Study Table	Furniture	3500	15
104	Office Chair	Furniture	2500	12
105	Story Book	Books	250	50
106	Notebook Set	Books	150	60
107	Sports Shoes	Fashion	2200	25
108	T-Shirt	Fashion	600	40

Mobile Phone added to cart successfully.

==========

E-COMMERCE MANAGEMENT SYSTEM

==========

1. Display Products
2. Search Product
3. Add Product to Cart
4. View Cart
5. Calculate Total Bill
6. Enter Customer Details
7. Display Customer Details
8. Place Order
9. Display Order Details
10. Display Sales Information
11. Show Graphs
12. Exit

------ YOUR CART ------

Name	Price	Qty	Subtotal
Mobile Phone	15000	3	45000

==========

E-COMMERCE MANAGEMENT SYSTEM

==========

1. Display Products
2. Search Product
3. Add Product to Cart
4. View Cart
5. Calculate Total Bill
6. Enter Customer Details
7. Display Customer Details
8. Place Order
9. Display Order Details
10. Display Sales Information
11. Show Graphs
12. Exit

------ BILL SUMMARY ------

Total Amount : Rs. 45000

Discount Applied : Rs. 4500.0

Final Amount : Rs. 40500.0

==========

E-COMMERCE MANAGEMENT SYSTEM

==========

1. Display Products
2. Search Product
3. Add Product to Cart
4. View Cart
5. Calculate Total Bill
6. Enter Customer Details
7. Display Customer Details
8. Place Order
9. Display Order Details
10. Display Sales Information
11. Show Graphs
12. Exit

Customer details saved successfully.

==========

E-COMMERCE MANAGEMENT SYSTEM

==========

1. Display Products
2. Search Product
3. Add Product to Cart
4. View Cart
5. Calculate Total Bill
6. Enter Customer Details
7. Display Customer Details
8. Place Order
9. Display Order Details
10. Display Sales Information
11. Show Graphs
12. Exit

------ BILL SUMMARY ------

Total Amount : Rs. 45000

Discount Applied : Rs. 4500.0

Final Amount : Rs. 40500.0

Order placed successfully! Your Order No. is 1

==========

E-COMMERCE MANAGEMENT SYSTEM

==========

1. Display Products
2. Search Product
3. Add Product to Cart
4. View Cart
5. Calculate Total Bill
6. Enter Customer Details

7. Display Customer Details

8. Place Order

9. Display Order Details

10. Display Sales Information

11. Show Graphs

12. Exit

------ ORDER DETAILS ------

Order No : 1

Customer : Arun

Amount : Rs. 40500.0

Items :

- Mobile Phone x 3

==========

E-COMMERCE MANAGEMENT SYSTEM

==========

1. Display Products

2. Search Product

3. Add Product to Cart

4. View Cart

5. Calculate Total Bill

6. Enter Customer Details

7. Display Customer Details

8. Place Order

9. Display Order Details

10. Display Sales Information

11. Show Graphs

12. Exit

------ CATEGORY WISE SALES ------

Electronics : 320 units

Furniture : 150 units

Books : 210 units

Fashion : 180 units

==========

E-COMMERCE MANAGEMENT SYSTEM

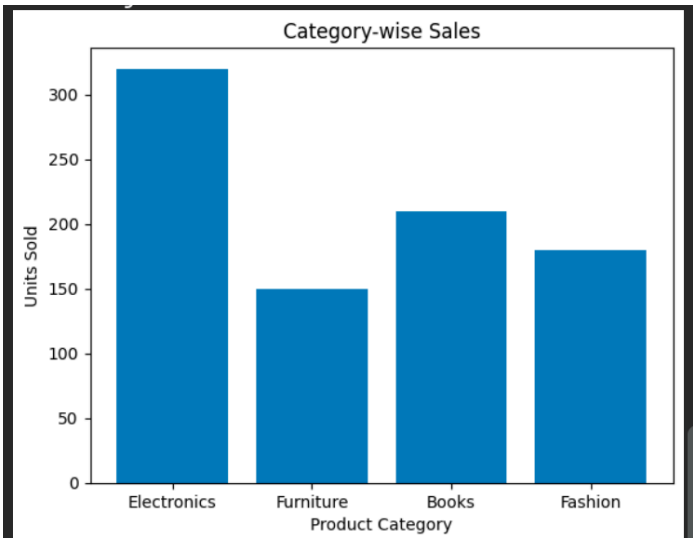
==========

1. Display Products
2. Search Product
3. Add Product to Cart
4. View Cart
5. Calculate Total Bill
6. Enter Customer Details
7. Display Customer Details
8. Place Order
9. Display Order Details
10. Display Sales Information
11. Show Graphs
12. Exit

---- GRAPH MENU ----

1. Bar Chart (Category wise Sales)
2. Line Graph (Month wise Sales)
3. Histogram (Order Value Distribution)
4. Back to Main Menu

Enter your choice: 1



==========

E-COMMERCE MANAGEMENT SYSTEM

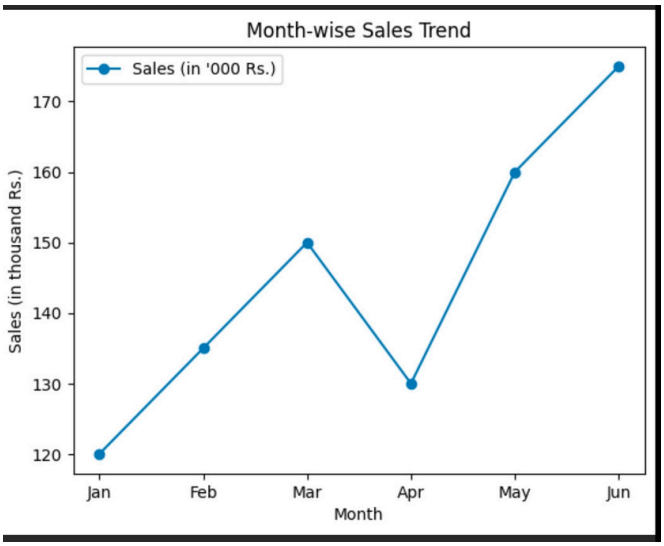
==========

1. Display Products
2. Search Product
3. Add Product to Cart
4. View Cart
5. Calculate Total Bill
6. Enter Customer Details
7. Display Customer Details
8. Place Order
9. Display Order Details
10. Display Sales Information
11. Show Graphs
12. Exit

---- GRAPH MENU ----

1. Bar Chart (Category wise Sales)
2. Line Graph (Month wise Sales)
3. Histogram (Order Value Distribution)
4. Back to Main Menu

Enter your choice: 2



==========

E-COMMERCE MANAGEMENT SYSTEM

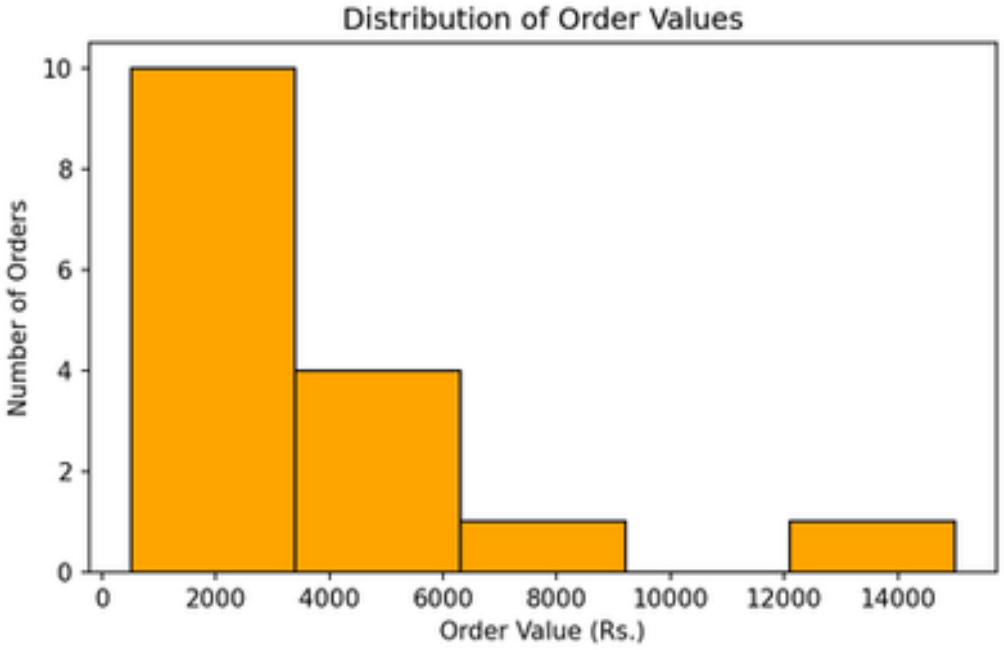
==========

1. Display Products
2. Search Product
3. Add Product to Cart
4. View Cart
5. Calculate Total Bill
6. Enter Customer Details
7. Display Customer Details
8. Place Order
9. Display Order Details
10. Display Sales Information
11. Show Graphs
12. Exit

---- GRAPH MENU ----

1. Bar Chart (Category wise Sales)
2. Line Graph (Month wise Sales)
3. Histogram (Order Value Distribution)
4. Back to Main Menu

Enter your choice: 3



==========

E-COMMERCE MANAGEMENT SYSTEM

==========

1. Display Products
2. Search Product
3. Add Product to Cart
4. View Cart
5. Calculate Total Bill
6. Enter Customer Details
7. Display Customer Details
8. Place Order
9. Display Order Details
10. Display Sales Information
11. Show Graphs
12. Exit

Enter your choice (1-12): 12

Thank you for shopping with us. Goodbye!

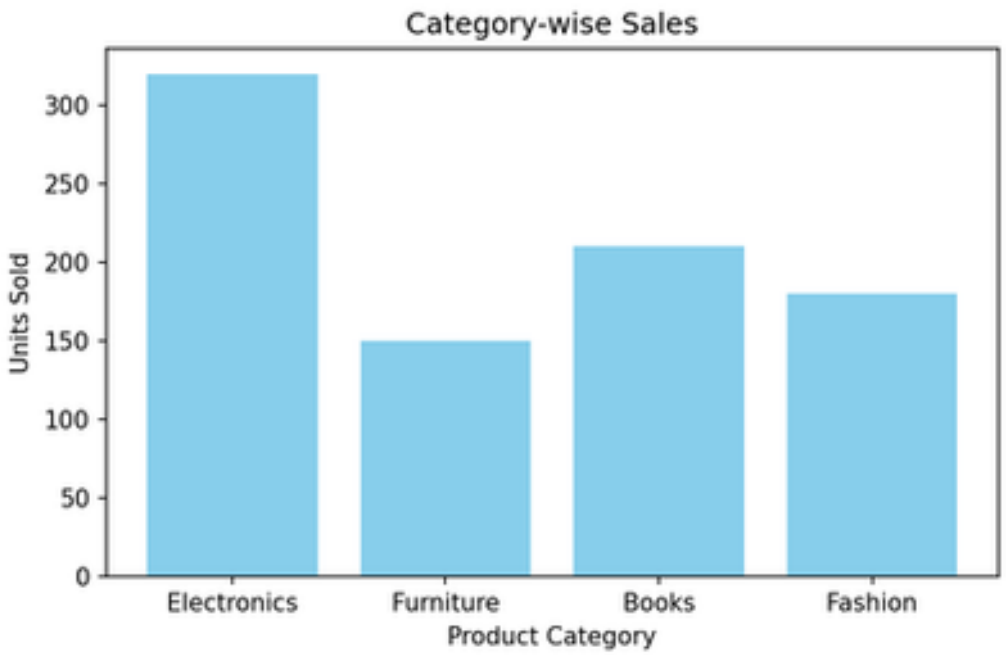
Data Visualization: Bar Chart

A bar chart is used to compare quantities across different categories using rectangular bars. In this project, a bar chart has been used to show category-wise sales (in units) for Electronics, Furniture, Books and Fashion.

Code Used:

```
categories = list(category_sales.keys())
sales = list(category_sales.values())

plt.bar(categories, sales, color="skyblue")
plt.title("Category-wise Sales")
plt.xlabel("Product Category")
plt.ylabel("Units Sold")
plt.savefig("bar_chart.png")
plt.show()
```



Result: The bar chart clearly shows that Electronics has the highest sales (320 units), followed by Books (210 units), Fashion (180 units) and Furniture (150 units). This helps the store understand which category of products is most popular among customers.

Data Visualization: Line Graph

A line graph is used to show how a value changes over a period of time. In this project, a line graph has been used to display the total sales (in thousand rupees) for each month from January to June.

Code Used:

```
plt.plot(month_names, month_sales, color="green",
         marker="o", label="Sales (in '000 Rs.)")
plt.title("Month-wise Sales Trend")
plt.xlabel("Month")
plt.ylabel("Sales (in thousand Rs.)")
plt.legend()
plt.savefig("line_graph.png")
plt.show()
```



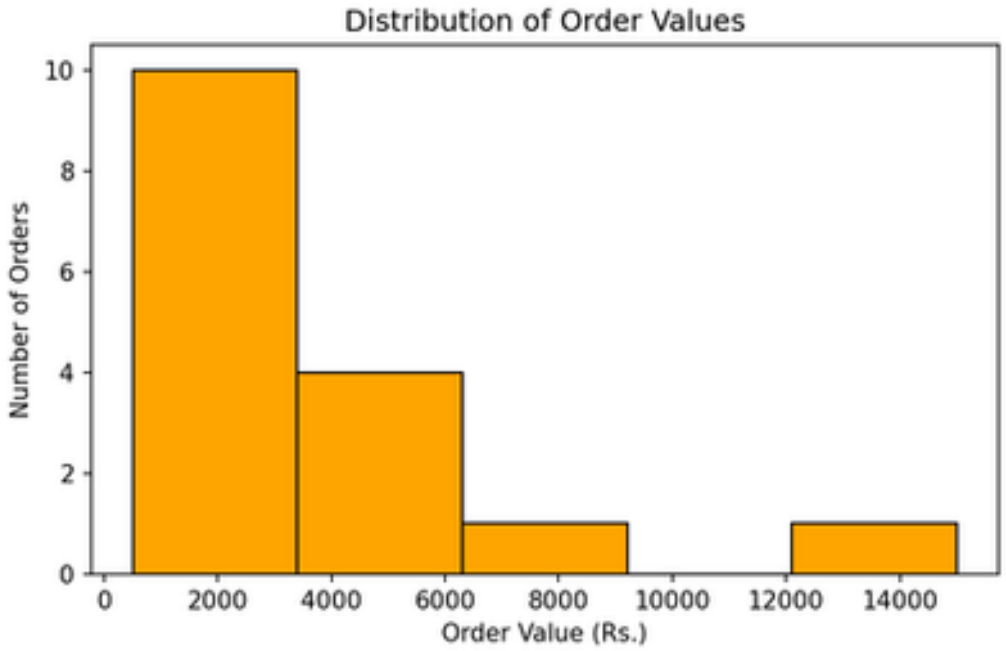
Result: The line graph shows that sales generally increased from January to June, with a small dip in April. The highest sales were recorded in June. This kind of graph helps in understanding sales trends over time.

Data Visualization: Histogram

A histogram is used to show the distribution of numerical data by grouping values into ranges (called bins). In this project, a histogram has been used to show how many past orders fall into different order-value ranges.

Code Used:

```
plt.hist(past_order_values, bins=5, color="orange",
         edgecolor="black")
plt.title("Distribution of Order Values")
plt.xlabel("Order Value (Rs.)")
plt.ylabel("Number of Orders")
plt.savefig("histogram.png")
plt.show()
```



Result: The histogram shows that most of the past orders have a relatively low order value, while only a few orders have very high values (such as Rs. 15,000). This kind of graph helps a business understand common spending patterns of its customers.

Testing

The program was tested manually with different sets of inputs to check its correctness. Some of the test cases are given below:

S.No.	Test Case	Expected Result	Actual Result
1	Search for an existing product	Product details displayed	As expected
2	Add product with quantity more than stock	Error message shown	As expected
3	Enter alphabets instead of Product ID	'Invalid input' message shown	As expected
4	Calculate bill with amount >= 20000	10% discount applied	As expected
5	Place order without entering customer details	Warning message shown	As expected
6	View cart when cart is empty	'Cart is empty' message shown	As expected
7	Enter invalid menu choice (e.g. 15)	'Invalid choice' message shown	As expected

Advantages

- Simple and easy to understand, menu-driven design.
- Automatically calculates bills and applies discounts.
- Maintains organized records of products, customers and orders.
- Provides visual insights into sales data using graphs.
- Built entirely using basic Python concepts, making it beginner-friendly.

Limitations

- Data is stored only in lists and dictionaries, so it is lost once the program is closed (no permanent storage).
- There is no login/authentication system for customers or admin.
- The system does not support real online payments.
- Only a limited number of sample products are available.

Future Scope

- The project can be extended by storing data permanently using files or databases.
- A graphical user interface (GUI) can be added to make it more user-friendly.
- Login systems for customers and admin can be introduced.
- More products, categories and payment options can be added.

Conclusion

This project, "E-Commerce Management System," successfully demonstrates how basic Python programming concepts taught in Class XII Informatics Practices can be used to build a simple, functional application similar to a real-world online store. Through this project, concepts like Lists, Dictionaries, Functions, conditional statements, loops and data visualization using Matplotlib were applied practically.

Working on this project helped in understanding how real-life problems, such as managing products, carts and orders, can be broken down into small, manageable functions. It also helped in understanding how data can be visualized meaningfully using simple graphs.

Bibliography

- NCERT/CBSE Informatics Practices Class XII Textbook.
- Python official documentation - https://docs.python.org
- Matplotlib official documentation - https://matplotlib.org
- www.geeksforgeeks.org - for reference on Python concepts.

- www.w3schools.com - for Python and Matplotlib syntax reference.